

Name: Natalie Poche
Date: September 25, 2026
UFID: 1533-9442

Report - Bootloader Development

1 Introduction

The purpose of this lab assignment is to develop raw binary bootloader images for both x86 and ARM processor architectures to understand the basics of boot routines and the services they provide. A primary objective of this project is to highlight the stark differences between ARM and x64 boot routines and their respective levels of hardware standardization. In this activity, bootloaders were written for the Intel i386 platform (using the 486 virtual CPU) and the ARM “Akita” platform. The developed bootloaders are designed to read a null-terminated string from a memory-mapped logging device and transmit this message to a serial Universal Asynchronous Receiver-Transmitter (UART) output device before halting execution.

2 Design

The design phase involved writing assembly source code and utilizing a GCC toolchain—specifically utilizing cross-compilation options for the ARM target—to build raw binary images via Makefiles. To test and verify the bootloaders, the QEMU command-line emulator and GNU Debugger (gdb-multiarch) were used. QEMU’s loader driver was explicitly used to simulate the mapping of flash ROM into the memory space.

On the x86 platform, the system architecture utilizes a standardized Basic Input-Output System (BIOS) loaded from ROM to initialize basic input and output devices. Because of this, the first stage bootloader does not need to worry about hardware initialization or loading the boot block (the first 512 bytes) into RAM, as the BIOS automatically handles loading it into address `0x7c00`. The logging device is mapped directly to the beginning of conventional memory at `0x500`. Furthermore, the UART devices (COM ports) on x86 are accessed via the data bus using a base port address (e.g., `0x03F8` for COM1) rather than being memory-mapped.

Conversely, the ARM architecture lacks a standardized firmware boot sequence and does not feature standardized address mapping for devices. Therefore, the first-stage bootloader must be explicitly developed for the exact hardware configuration. On the Akita platform, the bootloader begins execution at the trap table at address `0x000`, the memory-mapped log device is located at `0xA3F00000`, and the UART device is memory-mapped at `0x40100000`. Before data can be transmitted, the ARM bootloader must manually initialize the CPU by modifying the Current Program Status Register (CPSR) to set supervisor mode (`0x13`), disable THUMB instructions, and mask regular and fast IRQs.

3 Results and Discussion

Following successful builds, an investigative experiment was conducted utilizing different CPU emulations in QEMU to test the portability of the bootloader images.

x86 Experiment

After successfully running the bootloader on the i386 platform (486 CPU), the same bootloader image was executed using the x86_64 emulator (qemu64 CPU).

1. **Result:** The machine completed the task successfully.
2. **Reasoning:** This success was observed because x86 platforms are highly standardized. The 32-bit (and inherently 64-bit) x86 lines start in 16-bit Real Mode for backwards compatibility. Because the BIOS boot procedures, memory map ranges, and hardware data bus mapping for UART remain strictly standardized across x86 generations, the identical bootloader image executes flawlessly on both the 486 and qemu64 virtual CPUs.

ARM Experiment

After successfully running the bootloader on the “akita” platform, the same image was executed using the “raspi2” platform.

3. **Result:** The machine failed to complete the task successfully.
4. **Reasoning:** ARM processors do not have standardized address mapping for devices. The specific addresses hardcoded into the bootloader for the Akita platform—such as `0x40100000` for UART—do not correspond to the same device registers on the raspi2 platform.
5. **Platform Differences:** This differs significantly from how the x86 bootloader operates. While an x86 bootloader can rely on standard memory spaces (e.g., conventional memory starting at `0x500`) and standardized data bus ports, at least the first stage of an ARM bootloader must be developed explicitly for a specific hardware layout.

4 Conclusion

The successful development of x86 and ARM bootloaders highlights the structural differences in how architectures handle system initialization. The x86 boot process is greatly simplified by the standard BIOS, which performs necessary device initialization and reliably maps conventional memory and COM ports. In contrast, the ARM platform demands manual CPU initialization via the CPSR and necessitates querying hardware-specific memory-mapped registers, padding them by 4 bytes for reads and writes. The investigative experiment confirmed that while x86 boot blocks are broadly portable across processor generations due to backwards compatibility, ARM first-stage bootloaders are fundamentally tied to their specific hardware configurations.

A Project Deliverables

As required by the project specifications, a screencast demonstrating the Makefiles, build process, and QEMU emulation of both bootloaders is provided below.

Screencast Link: <https://youtube.com/unlisted-link-placeholder>

B Code Snippets

Below are the source assembly files and build automation script imported directly from external code files using the `\lstinputlisting` command.

Listing 1: x86 Assembly Bootloader Source (x86_boot.s)

```
.code16
.global _start

_start:
    /* Set string source index to log device memory map */
    mov $0x500, %si

print_loop:
    /* Load byte from %ds:%si into %al and increment %si */
    lodsb
    test %al, %al
    jz halt_cpu

wait_uart:
    /* Read LSR to check UART status */
    # Replace direct immediate access:
    # inb $0x3fd, %al Incorrect (truncates to 0xfd)

    # With register-based I/O:
    mov $0x3fd, %dx
    inb %dx, %al
    test $0x20, %al
    jz wait_uart

send_uart:
    /* Retrieve the character and output to THR */
    mov -1(%si), %al
    # Replace direct immediate access:
    # outb %al, $0x3f8 Incorrect (truncates to 0xf8)

    # With register-based I/O:
    mov $0x3f8, %dx
    outb %al, %dx
    jmp print_loop

halt_cpu:
    hlt
    jmp halt_cpu

/* Pad remainder of boot sector and add boot signature */
.org 510
.word 0xaa55
```

Listing 2: ARM Assembly Bootloader Source (arm_boot.s)

```
.global _start
```

```

_start:
    /* 1. CPU Initialization */
    mrs r0, cpsr
    bic r0, r0, #0xff
    orr r0, r0, #0xd3
    msr cpsr_c, r0

    /* 2. Setup Memory Addresses */
    ldr r1, =0xA3F00000    /* Log device address */
    ldr r2, =0x40100000    /* UART THR (Base) */
    ldr r3, =0x40100014    /* UART LSR */

print_loop:
    /* Load byte and post-increment log pointer */
    ldrb r4, [r1], #1
    cmp r4, #0
    beq halt_cpu

wait_uart:
    /* Read LSR and check if THR is empty (0x20) */
    ldr r5, [r3]
    tst r5, #0x20
    beq wait_uart

send_uart:
    /* Write character to THR */
    strb r4, [r2]
    b print_loop

halt_cpu:
    b halt_cpu

```

Listing 3: Build Automation Script (Makefile)

```

all: i386_image.bin akita_image.bin

# x86 Build Process
i386_image.bin: x86_image.s
#       # as -march=i386 x86_image.s -o x86.o
#       as --32 -march=i386 x86_image.s -o x86.o
#       ld -Ttext 0x7c00 --oformat binary x86.o -o i386_image.bin
#       ld -m elf_i386 -Ttext 0x7c00 --oformat binary x86.o -o i386_image.bin

# ARM Build Process
akita_image.bin: akita_image.s
#       arm-none-eabi-as -march=armv4t akita_image.s -o arm.o
#       arm-none-eabi-ld -Ttext 0x0 arm.o -o arm.elf
#       arm-none-eabi-objcopy -O binary --only-section=.text arm.elf akita_image.bin

clean:
    rm -f *.o *.elf *.bin

```